



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Институт (Филиал) № 8 «Компьютерные науки и прикладная математика» Кафедра 806

Группа М8О-408Б-22 Направление подготовки 01.03.02 «Прикладная математика и информатика»

Профиль Информатика

Квалификация: бакалавр

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА

На тему: Программный комплекс для оценки эффективности алгоритмов управления мульти-роботизированными системами

Автор ВКРБ: Галкин Алексей Дмитриевич ()

Руководитель: Киндинова Виктория Валерьевна ()

Консультант: ()

Консультант: ()

Рецензент: ()

К защите допустить

Заведующий кафедрой № 806 Крылов Сергей Сергеевич ()

____ мая 2026 года

РЕФЕРАТ

Выпускная квалификационная работа бакалавра состоит из 55 страниц, 11 рисунков, 5 таблиц, 19 использованных источников, 1 приложения.

МУЛЬТИ-РОБОТИЗИРОВАННЫЕ СИСТЕМЫ, MAPF, MRTA, ПОИСК БЕСКОНФИКТНЫХ МАРШРУТОВ, РАСПРЕДЕЛЕНИЕ ЗАДАЧ, СРАВНИТЕЛЬНЫЙ АНАЛИЗ АЛГОРИТМОВ, ПРОГРАММНЫЙ КОМПЛЕКС

Объектом разработки является кроссплатформенный программный комплекс для оценки эффективности алгоритмов управления мульти-роботизированными системами.

Цель работы — создание программного комплекса, позволяющего проводить тестирование и сравнительный анализ алгоритмов поиска бесконфликтных маршрутов (MAPF) и распределения задач (MRTA) при различных конфигурациях среды.

Методы проведения работы: анализ и классификация алгоритмов на основе научной литературы; объектно-ориентированное проектирование с применением паттерна «Стратегия» для обеспечения модульности и расширяемости; реализация алгоритмов на Python с графическим интерфейсом на Pygame; автоматизированное пакетное тестирование с варьированием параметров и статистической обработкой результатов.

В ходе работы реализованы восемь MAPF-алгоритмов (CBS, CBS-SIPP, ECBS, EECBS, M*, CA*, WHCA*, PIBT) и пять MRTA-алгоритмов (венгерский, жадный, последовательный и комбинаторный аукционы, метод минимальной стоимости потока). Разработан графический интерфейс с редактором карт, визуализацией симуляции и режимом пакетного тестирования с автогенерацией графиков. Обеспечена кроссплатформенная сборка в автономный исполняемый файл для Windows, Linux и macOS.

Комплекс устраняет разрозненность существующих инструментов, объединяя алгоритмы MAPF и MRTA в единой среде с визуализацией и пакетным тестированием. Это обеспечивает обоснованный выбор алгоритмов по объективным метрикам. Комплекс применим в научных исследованиях по координации групп агентов, прототипировании систем складской логистики, автономного транспорта и спасательных операций.

СОДЕРЖАНИЕ

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ	6
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	7
ВВЕДЕНИЕ	8
1 ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ	11
1.1 Мульти-роботизированные системы	11
1.2 Классификация задач управления мульти-роботизированными системами	12
1.3 Задача распределения задач в мульти-роботизированных системах	12
1.3.1 Формальная постановка задачи MRTA	12
1.3.2 Классификация MRTA-алгоритмов	13
1.3.3 Венгерский алгоритм	14
1.3.4 Жадный алгоритм	14
1.3.5 Случайное распределение задач	15
1.3.6 Алгоритм последовательного аукциона	15
1.3.7 Алгоритм минимальной стоимости потока	16
1.3.8 Комбинаторный аукцион	16
1.4 Задача поиска бесконфликтных маршрутов	17
1.4.1 Формальная постановка задачи MAPF	17
1.4.2 Классификация MAPF-алгоритмов	18
1.4.3 Алгоритм A* с расширением по времени	19
1.4.4 Conflict-Based Search (CBS)	20
1.4.5 Enhanced CBS (ECBS)	21
1.4.6 Explicit Estimation CBS (EECBS)	22
1.4.7 M* (Subdimensional Expansion)	22
1.4.8 Cooperative A* (CA*)	23
1.4.9 Windowed Hierarchical CA* (WHCA*)	24
1.4.10 Priority Inheritance with Backtracking (PIBT)	24
1.5 Интеграция MRTA и MAPF	25
2 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО КОМПЛЕКСА	27
2.1 Выбор стека технологий	27
2.2 Архитектура программного комплекса	28
2.3 Базовые классы алгоритмов	29
2.3.1 Класс MAPFPlanner	29

2.3.2	Класс MRTAPlanner	30
2.4	Модель данных	31
2.4.1	Класс World	31
2.4.2	Класс Agent	31
2.4.3	Класс Task	31
2.4.4	Класс SceneData	31
2.5	Форматы входных файлов сценариев	31
2.5.1	Внутренний формат JSON	32
2.5.2	Формат Moving AI .map	32
2.5.3	Формат Moving AI .scen	32
2.6	Система регистрации алгоритмов	33
2.7	Симулятор: планирование и выполнение	33
2.7.1	Этап планирования	33
2.7.2	Этап выполнения	34
2.7.3	Сбор метрик	34
2.8	Подсистема пакетного тестирования	35
2.8.1	Конфигурация эксперимента	35
2.8.2	Генерация сценариев	36
2.8.3	Использование файлов .scen в пакетном режиме	36
2.8.4	Запуск и вывод результатов	37
3	РЕАЛИЗАЦИЯ И РЕЗУЛЬТАТЫ	39
3.1	Пользовательский интерфейс	39
3.1.1	Главное меню	39
3.1.2	Редактор карт	40
3.1.3	Экран выбора алгоритма	40
3.1.4	Визуализация симуляции	41
3.1.5	Батч-режим	41
3.2	Сравнительный анализ алгоритмов	42
3.2.1	Методика эксперимента	42
3.2.2	Анализ MAPF-алгоритмов	43
3.2.3	Анализ MRTA-алгоритмов	45
3.2.4	Результаты пакетного тестирования	46
3.3	Обсуждение результатов	49
	ЗАКЛЮЧЕНИЕ	51
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	53

ПРИЛОЖЕНИЕ А Репозиторий проекта	55
--	----

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящей выпускной квалификационной работе бакалавра применяют следующие термины с соответствующими определениями:

CA* — Cooperative A* — алгоритм кооперативного поиска пути

CBS — Conflict-Based Search — алгоритм поиска на основе конфликтов

CT — Constraint Tree — дерево ограничений в алгоритме CBS

ECBS — Enhanced Conflict-Based Search — ограниченно субоптимальное расширение CBS с фокальным поиском на обоих уровнях

EECBS — Explicit Estimation CBS — развитие ECBS с онлайн-эвристикой явной оценки стоимости разрешения конфликтов

M* — M-Star (Subdimensional Expansion) — оптимальный алгоритм MAPF, расширяющий пространство поиска до полного совместного только в точках возникновения конфликтов

Makespan — метрика времени завершения миссии: максимальное из времён прибытия всех агентов в целевые позиции

MAPF — Multi-Agent Path Finding — задача поиска бесконфликтных маршрутов для группы агентов

MRTA — Multi-Robot Task Assignment — задача распределения задач между роботами

PIBT — Priority Inheritance with Backtracking — децентрализованный онлайн-алгоритм MAPF с динамическим наследованием приоритетов и откатом

SIPP — Safe Interval Path Planning — алгоритм поиска пути с использованием безопасных временных интервалов вместо перебора отдельных временных шагов

SOC — Sum of Costs — метрика суммарной стоимости путей всех агентов

WHCA* — Windowed Hierarchical Cooperative A* — CA* с ограниченным окном планирования

MPC — мульти-роботизированная система

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящей выпускной квалификационной работе бакалавра применяют следующие сокращения и обозначения:

BFS — Breadth-First Search
CA* — Cooperative A*
CBS — Conflict-Based Search
CSV — Comma-Separated Values
CT — Constraint Tree
ECBS — Enhanced Conflict-Based Search
EECBS — Explicit Estimation Conflict-Based Search
IA — Instantaneous Assignment
JSON — JavaScript Object Notation
M* — M-Star, Subdimensional Expansion
MAPF — Multi-Agent Path Finding
MR — Multi-Robot
MRTA — Multi-Robot Task Assignment
MT — Multi-Task
NP — Nondeterministic Polynomial
PIBT — Priority Inheritance with Backtracking
SIPP — Safe Interval Path Planning
SOC — Sum of Costs
SR — Single-Robot
ST — Single-Task
TA — Time-Extended Assignment
WHCA* — Windowed Hierarchical Cooperative A*
MPC — мульти-роботизированная система

ВВЕДЕНИЕ

Развитие робототехники и искусственного интеллекта в последние десятилетия привело к широкому распространению мульти-роботизированных систем (МРС) в различных областях деятельности человека. Группы роботов, действующих совместно, применяются в складской логистике, автономном транспорте, поисково-спасательных операциях, сельском хозяйстве и многих других сферах [1; 2]. Использование нескольких автономных агентов вместо одного позволяет повысить производительность за счёт параллельного выполнения задач, обеспечить отказоустойчивость системы и масштабируемость решений. Координация множества автономных агентов, работающих в общем пространстве, представляет собой одну из ключевых задач современной робототехники.

Управление мульти-роботизированными системами включает два фундаментальных класса задач: распределение задач между роботами (Multi-Robot Task Assignment, MRTA) и поиск бесконфликтных маршрутов для группы агентов (Multi-Agent Path Finding, MAPF). Задача MRTA определяет, какой робот выполняет какую задачу, а задача MAPF обеспечивает построение маршрутов движения роботов к назначенным целям таким образом, чтобы агенты не сталкивались друг с другом. Для решения каждой из этих задач разработано значительное число алгоритмов, существенно различающихся по архитектуре, вычислительной сложности, оптимальности получаемых решений и области применимости [3; 4].

Актуальность работы обусловлена растущей потребностью в инструментах для объективного сравнения алгоритмов управления МРС. Существующие реализации алгоритмов MAPF и MRTA, как правило, разрозненны, выполнены в различных программных средах и на разных языках программирования, что затрудняет их прямое сравнение в единых условиях. Кроме того, большинство доступных инструментов ориентированы на исследователей и не предоставляют интуитивного графического интерфейса для настройки экспериментов, визуализации работы алгоритмов и автоматизированного сбора метрик. Выбор подходящего алгоритма для конкретной прикладной задачи остаётся нетривиальной проблемой, поскольку эффективность каждого алгоритма существенно зависит от параметров среды: размера карты, количества роботов, плотности препятствий и характера

выполняемых задач.

Целью данной работы является создание кроссплатформенного программного комплекса для тестирования и сравнения эффективности алгоритмов управления мульти-роботизированными системами.

Для достижения поставленной цели необходимо решить следующие задачи:

- 1) провести анализ существующих алгоритмов решения задач MAPF и MRTA, их классификацию и особенности применения;
- 2) разработать архитектуру программного комплекса, обеспечивающую модульность и расширяемость;
- 3) реализовать алгоритмы поиска бесконфликтных маршрутов: CBS, CBS-SIPP, ECBS, EECBS, M*, CA*, WHCA*, PIBT;
- 4) реализовать алгоритмы распределения задач: венгерский алгоритм, жадный алгоритм, последовательный аукцион, минимальная стоимость потока, комбинаторный аукцион;
- 5) разработать графический интерфейс пользователя, включающий редактор карт, панель выбора алгоритмов и визуализацию симуляции;
- 6) реализовать режим пакетного тестирования для автоматизированного сравнительного анализа алгоритмов при различных параметрах среды;
- 7) обеспечить кроссплатформенность программного комплекса;
- 8) провести сравнительный анализ реализованных алгоритмов по ключевым метрикам эффективности.

Объектом исследования является программный комплекс для оценки эффективности алгоритмов управления мульти-роботизированными системами. Предметом исследования являются алгоритмы управления мульти-роботизированными системами и их сравнительная эффективность при различных конфигурациях среды и количестве агентов.

Результатом работы является кроссплатформенный программный комплекс на языке Python с графическим интерфейсом на базе библиотеки Pygame, реализующий восемь MAPF-алгоритмов и пять MRTA-алгоритмов. Комплекс предоставляет интерактивную среду для создания сценариев на сеточных картах, визуализации работы алгоритмов в режиме реального времени и проведения автоматизированного пакетного сравнительного

анализа с генерацией отчётов и графиков по ключевым метрикам: времени выполнения (makespan), суммарной стоимости путей (SOC), времени планирования и потреблению памяти. С помощью PyInstaller обеспечена сборка в автономный исполняемый файл для операционных систем Windows, Linux и macOS.

1 ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Мульти-роботизированные системы

Мульти-роботизированная система (МРС) представляет собой совокупность нескольких автономных роботов, функционирующих в общей среде и взаимодействующих между собой для достижения заданных целей [2]. В отличие от одиночных роботизированных решений, МРС обладают рядом существенных преимуществ.

Во-первых, параллелизм: несколько роботов способны одновременно выполнять различные задачи, что сокращает общее время выполнения миссии. Во-вторых, отказоустойчивость: выход из строя одного агента не приводит к полной остановке системы, поскольку оставшиеся роботы могут перераспределить задачи. В-третьих, масштабируемость: производительность системы может быть увеличена добавлением новых роботов без изменения управляющей архитектуры.

Практические применения МРС охватывают широкий спектр областей. В складской логистике системы автономных мобильных роботов обеспечивают перемещение товаров между стеллажами и зонами комплектации. Компания Amazon (ранее Kiva Systems) продемонстрировала возможность координации сотен роботов на одном складе [1]. В области автономного транспорта группы беспилотных автомобилей требуют согласованного планирования маршрутов для предотвращения столкновений. В поисково-спасательных операциях группы дронов или наземных роботов позволяют параллельно обследовать обширные территории, сокращая время реагирования. В сельском хозяйстве флотилии роботов используются для автоматизации посева, мониторинга и сбора урожая [5].

Основными задачами, возникающими при управлении МРС, являются: определение того, какие задачи выполняет каждый робот, и планирование маршрутов движения роботов таким образом, чтобы они не мешали друг другу. Эти задачи формализуются как Multi-Robot Task Assignment (MRTA) и Multi-Agent Path Finding (MAPF) соответственно.

1.2 Классификация задач управления мульти-роботизированными системами

Задачи управления MPC можно разделить на два основных класса, каждый из которых отвечает на свой вопрос:

- MRTA (Multi-Robot Task Assignment) — задача распределения задач: определяет, какой робот выполняет какую задачу. Результатом является назначение задач роботам с минимизацией выбранной целевой функции (суммарного расстояния, времени и т. д.);
- MAPF (Multi-Agent Path Finding) — задача поиска бесконфликтных маршрутов: определяет, как каждый робот добирается до назначенной цели, обеспечивая отсутствие столкновений между агентами. Результатом является набор путей для всех агентов [3; 4].

В реальных системах эти задачи тесно связаны: сначала MRTA-алгоритм назначает задачи роботам, затем MAPF-алгоритм строит бесконфликтные маршруты к назначенным целям. При этом качество решения задачи распределения существенно зависит от навигационных возможностей среды, а эффективность маршрутизации определяется назначенными целевыми позициями.

1.3 Задача распределения задач в мульти-роботизированных системах

1.3.1 Формальная постановка задачи MRTA

Задача MRTA формулируется следующим образом. Пусть дано множество роботов $R = \{r_1, r_2, \dots, r_n\}$ и множество задач $T = \{t_1, t_2, \dots, t_m\}$. Каждая задача t_j характеризуется целевой позицией на карте. Для каждой пары робот–задача определена функция стоимости $c(r_i, t_j)$, отражающая затраты на выполнение задачи t_j роботом r_i (как правило, это расстояние между текущей позицией робота и целевой позицией задачи).

Требуется найти назначение σ , сопоставляющее каждому роботу подмножество задач, минимизируя суммарную стоимость:

$$\min_{\sigma} \sum_{i=1}^n \sum_{t_j \in \sigma(r_i)} c(r_i, t_j), \quad (1)$$

при условии, что каждая задача назначена ровно одному роботу: $\sigma(r_i) \cap \sigma(r_k) = \emptyset$ для $i \neq k$ и $\bigcup_{i=1}^n \sigma(r_i) = T$.

1.3.2 Классификация MRTA-алгоритмов

Б. П. Джерки и М. Дж. Матарич предложили таксономию MRTA-задач по трём измерениям [4]:

- ST vs MT (Single-Task vs Multi-Task) — может ли робот выполнять одну или несколько задач одновременно;
- SR vs MR (Single-Robot vs Multi-Robot) — требуется ли для выполнения задачи один или несколько роботов;
- IA vs TA (Instantaneous Assignment vs Time-extended Assignment) — выполняется ли назначение однократно или с учётом временной динамики.

Г. А. Корса, Э. Стенц и М. Б. Диас расширили данную таксономию, добавив межзадачные зависимости и утилитарные функции [6]. В рамках настоящей работы рассматривается наиболее распространённый класс задач ST-SR — каждый робот выполняет по одной задаче за раз, и каждая задача требует одного робота.

Помимо этого, различают два режима назначения задач:

- Online-режим — робот получает одну задачу; после её завершения планировщик назначает следующую. Это соответствует варианту IA;
- Queue-режим — планировщик сразу выстраивает полную цепочку задач для каждого робота, что соответствует варианту TA.

По подходу к решению MRTA-алгоритмы подразделяются на:

- оптимизационные — формулируют задачу как задачу оптимизации и находят точное или приближённое решение (венгерский алгоритм, алгоритм минимальной стоимости потока);
- эвристические — используют простые правила для быстрого построения допустимого назначения (жадный алгоритм);
- рыночные (market-based) — моделируют процесс торгов, в котором роботы конкурируют за задачи (последовательный аукцион,

комбинаторный аукцион) [7].

1.3.3 Венгерский алгоритм

Венгерский алгоритм решает задачу о назначениях — частный случай MRTA, в котором число роботов равно числу задач и требуется найти взаимно однозначное соответствие с минимальной суммарной стоимостью [8].

Формально, задача о назначениях записывается как:

$$\min_{\sigma \in S_n} \sum_{i=1}^n c_{i,\sigma(i)}, \quad (2)$$

где S_n — множество всех перестановок $\{1, 2, \dots, n\}$, а $c_{i,j}$ — элемент матрицы стоимостей $C \in \mathbb{R}^{n \times n}$.

Алгоритм основан на теореме Кёнига и последовательно приводит матрицу стоимостей к виду, в котором оптимальное назначение соответствует набору нулевых элементов, покрывающему все строки и столбцы. Вычислительная сложность алгоритма составляет $O(n^3)$, что делает его эффективным для задач умеренного размера.

В контексте MPC венгерский алгоритм работает в online-режиме: за одну итерацию каждому роботу назначается ровно одна задача, а после завершения задачи алгоритм перезапускается для оставшихся незавершённых задач. Матрица стоимостей формируется на основе манхэттенского расстояния между текущими позициями роботов и целевыми позициями задач.

1.3.4 Жадный алгоритм

Жадный алгоритм назначения задач представляет собой простейший эвристический подход к решению задачи MRTA. На каждом шаге алгоритм выбирает пару робот–задача (r_i, t_j) с минимальной стоимостью $c(r_i, t_j)$ среди всех ещё не назначенных задач и свободных роботов, и назначает задачу t_j роботу r_i .

Вычислительная сложность жадного алгоритма составляет $O(n \cdot m)$, где n — число роботов, m — число задач. Алгоритм не гарантирует оптимальности решения, однако обеспечивает быстрое построение допустимого назначения, что может быть критично в системах реального времени.

Аналогично венгерскому алгоритму, жадный алгоритм работает в online-режиме: после завершения задачи роботом алгоритм перезапускается для назначения следующей задачи.

1.3.5 Случайное распределение задач

Случайное распределение задач представляет собой простейший эвристический алгоритм, в котором назначение строится без учёта функции стоимости. Множества роботов и задач независимо перемешиваются с помощью генератора псевдослучайных чисел с фиксированным начальным значением (*seed*), после чего сопоставляются попарно: i -й после перемешивания робот получает i -ю после перемешивания задачу. При неравных мощностях множеств часть роботов остаётся без задач либо часть задач — без исполнителей.

Вычислительная сложность алгоритма составляет $O(\max(n, m))$, что делает его наиболее быстрым среди рассмотренных. Алгоритм не оптимизирует ни одну из метрик качества и заведомо уступает остальным MRTA-алгоритмам по суммарной стоимости назначения. В то же время его применение позволяет получить нижнюю границу качества и, что более важно, выделить вклад MAPF-планировщика в итоговые метрики: при сравнении различных MAPF-алгоритмов на одних и тех же случайных назначениях наблюдаемые различия определяются исключительно качеством навигационного планирования.

1.3.6 Алгоритм последовательного аукциона

Алгоритм последовательного аукциона (Sequential Auction) реализует рыночный подход к распределению задач [7]. Задачи выставляются на торги поочерёдно, и роботы подают заявки (ставки) на их выполнение. Задача назначается роботу с минимальной ставкой.

В реализации, рассматриваемой в данной работе, алгоритм работает в queue-режиме и строит полные цепочки задач для каждого робота. Ставка робота рассчитывается как стоимость добавления задачи в конец текущей цепочки: расстояние от виртуальной позиции (положения, в котором робот окажется после выполнения последней задачи в цепочке) до новой задачи. После назначения задачи виртуальная позиция робота обновляется.

Порядок выставления задач определяется разницей между двумя лучшими ставками (bid gap): первыми разыгрываются задачи, по которым расхождение ставок максимально, что уменьшает вероятность субоптимального назначения.

1.3.7 Алгоритм минимальной стоимости потока

Алгоритм минимальной стоимости потока (Min-Cost Flow) формулирует задачу MRTA как задачу нахождения потока минимальной стоимости в сети [9]. Строится двудольный граф, в котором вершины одной доли представляют роботов, а вершины другой — задачи. Рёбра имеют пропускную способность 1 и стоимость, равную расстоянию между роботом (или его виртуальной позицией) и задачей.

В реализации используется итеративный подход для построения цепочек задач (queue-режим): на каждой итерации решается задача минимальной стоимости потока для текущего «слоя» назначений, после чего виртуальные позиции роботов обновляются, и процесс повторяется для оставшихся задач. Решение каждой подзадачи потока выполняется алгоритмом последовательного кратчайшего пути (Successive Shortest Paths).

1.3.8 Комбинаторный аукцион

Комбинаторный аукцион (Combinatorial Auction) является расширением последовательного аукциона, в котором роботы подают заявки не на отдельные задачи, а на наборы (связки) задач [7]. Это позволяет учитывать синергию между задачами: выполнение двух близко расположенных задач одним роботом может быть более эффективным, чем их назначение разным роботам.

В реализации, рассматриваемой в данной работе, алгоритм работает в два этапа. На первом этапе используется жадное построение цепочек на основе сожаления (regret-based greedy): для каждой задачи вычисляется разница между стоимостью лучшего и второго лучшего назначения, и задачи с максимальным «сожалением» назначаются в первую очередь. На втором этапе выполняется локальная оптимизация: алгоритм пытается улучшить решение путём перемещения задач между цепочками роботов и обмена задачами между парами роботов, если это уменьшает суммарную стоимость.

1.4 Задача поиска бесконфликтных маршрутов

1.4.1 Формальная постановка задачи MAPF

Задача Multi-Agent Path Finding (MAPF) формулируется следующим образом [3]. Пусть дан неориентированный граф $G = (V, E)$, представляющий среду (в случае сеточной карты вершины соответствуют свободным клеткам, а рёбра — допустимым переходам между соседними клетками). Задано множество агентов $A = \{a_1, a_2, \dots, a_k\}$, для каждого из которых определена начальная позиция $s_i \in V$ и целевая позиция $g_i \in V$.

Путь агента a_i представляет собой последовательность вершин $\pi_i = (v_0^i, v_1^i, \dots, v_{T_i}^i)$, где $v_0^i = s_i$, $v_{T_i}^i = g_i$, и для каждого $t \in \{0, \dots, T_i - 1\}$ выполняется $(v_t^i, v_{t+1}^i) \in E$ (агент перемещается по ребру) или $v_t^i = v_{t+1}^i$ (агент ожидает на месте). Здесь T_i — время прибытия агента a_i в целевую вершину.

В литературе по задаче MAPF [3] различают пять основных типов конфликтов между планами агентов:

- вершинный конфликт (vertex conflict) — два агента a_i и a_j занимают одну вершину в один момент времени: $v_t^i = v_t^j$ при $i \neq j$;
- рёберный конфликт (edge conflict) — два агента проходят по одному и тому же ребру в одном направлении в один и тот же момент времени: $v_t^i = v_t^j$ и $v_{t+1}^i = v_{t+1}^j$ при $i \neq j$. В моделях, в которых каждая вершина может содержать не более одного агента, эта ситуация полностью поглощается вершинным конфликтом в момент t и не выделяется отдельно;
- обменный конфликт (swapping conflict) — два агента меняются позициями за один шаг: $v_t^i = v_{t+1}^j$ и $v_t^j = v_{t+1}^i$ при $i \neq j$;
- конфликт следования (following conflict) — один из агентов занимает вершину, которую другой только что освободил: $v_{t+1}^i = v_t^j$ при $i \neq j$;
- циклический конфликт (cycle conflict) — множество из трёх и более агентов одновременно совершает циклическую перестановку позиций: $v_{t+1}^{i_1} = v_t^{i_2}, v_{t+1}^{i_2} = v_t^{i_3}, \dots, v_{t+1}^{i_{k-1}} = v_t^{i_k}, v_{t+1}^{i_k} = v_t^{i_1}$.

Перечисленные определения соответствуют различным моделям одновременности ходов и физическим допущениям о роботах. В классической модели MAPF с одновременными дискретными ходами, принятой в большинстве работ по CBS [10; 11] и в эталонных наборах данных

Moving AI [12], физические столкновения между агентами могут порождать только вершинные и обменные конфликты: в моменты t и $t + 1$ агенты, не вовлечённые в указанные два типа, находятся в попарно различных вершинах. Конфликты следования и циклические конфликты в данной модели допустимы и приобретают значение лишь в более строгих постановках задачи — при последовательном исполнении ходов, при учёте физических размеров роботов либо при анализе устойчивости к задержкам в исполнении. В настоящей работе принята классическая модель, поэтому реализованный планировщик детектирует и разрешает конфликты двух типов: вершинные и обменные.

Набор путей $\Pi = \{\pi_1, \dots, \pi_k\}$ называется решением задачи MAPF, если он не содержит конфликтов. Для оценки качества решения используются две основные метрики:

$$\text{Makespan} = \max_{i \in \{1, \dots, k\}} T_i, \quad (3)$$

$$\text{SOC (Sum of Costs)} = \sum_{i=1}^k T_i. \quad (4)$$

Makespan отражает общее время до завершения миссии всеми агентами, тогда как SOC отражает суммарные затраты ресурсов (шагов) всех агентов. Задача нахождения оптимального решения MAPF (по любой из метрик) является NP-трудной, что мотивирует разработку как точных алгоритмов с приемлемой практической сложностью, так и приближённых алгоритмов [13].

1.4.2 Классификация MAPF-алгоритмов

Существующие MAPF-алгоритмы классифицируются по нескольким критериям [3].

1) По архитектуре управления:

- централизованные — единый планировщик строит пути для всех агентов одновременно, располагая полной информацией о среде (CBS, ECBS, EECBS, M*);
- децентрализованные — агенты планируют пути независимо или последовательно, обмениваясь

ограниченной информацией (CA*, WHCA*, PIBT).

2) По оптимальности:

- оптимальные — гарантируют нахождение решения с минимальной стоимостью (CBS, M*);
- ограниченно субоптимальные — гарантируют, что стоимость решения не превышает оптимальную более чем в w раз (ECBS, EECBS);
- неоптимальные — не дают гарантий оптимальности, но работают значительно быстрее (CA*, WHCA*, PIBT).

3) По режиму работы:

- offline — все пути строятся до начала движения агентов (CBS, ECBS, EECBS, M*, CA*, WHCA*);
- online — пути строятся или корректируются в процессе движения агентов (PIBT).

В таблице 1 приведена сводная классификация реализованных алгоритмов.

Таблица 1 – Классификация реализованных MAPF-алгоритмов

Алгоритм	Архитектура	Оптимальность	Режим
CBS (A*)	централиз.	оптимальный	offline
CBS (SIPP)	централиз.	оптимальный	offline
ECBS	централиз.	огр. субоптим.	offline
EECBS	централиз.	огр. субоптим.	offline
M*	централиз.	оптимальный	offline
CA*	децентрал.	неоптимальный	offline
WHCA*	децентрал.	неоптимальный	offline
PIBT	децентрал.	неоптимальный	online

1.4.3 Алгоритм A* с расширением по времени

Алгоритм A* является фундаментальным алгоритмом поиска кратчайшего пути в графе и лежит в основе большинства MAPF-алгоритмов [14]. В контексте задачи MAPF используется его модификация с расширением по времени (time-expanded A*), в которой состояние агента определяется не только позицией, но и моментом времени:

(v, t) , где $v \in V$ — текущая вершина, $t \in \mathbb{N}_0$ — текущий временной шаг.

Функция оценки каждого состояния определяется как:

$$f(v, t) = g(v, t) + h(v), \quad (5)$$

где $g(v, t) = t$ — стоимость достижения состояния (v, t) от начального состояния $(s_i, 0)$,

$h(v)$ — эвристическая оценка стоимости пути от v до цели g_i .

В качестве эвристики используется манхэттенское расстояние:

$$h(v) = |x_v - x_{g_i}| + |y_v - y_{g_i}|, \quad (6)$$

где (x_v, y_v) и (x_{g_i}, y_{g_i}) — координаты вершины v и целевой вершины g_i соответственно.

Алгоритм A^* с расширением по времени поддерживает множество ограничений (reservations) — набор состояний (v, t) , которые агент не может занимать (зарезервированы другими агентами). Это позволяет планировать пути с учётом уже построенных путей других агентов.

1.4.4 Conflict-Based Search (CBS)

Алгоритм CBS (Conflict-Based Search) является оптимальным централизованным алгоритмом решения задачи MAPF [10]. CBS использует двухуровневую архитектуру поиска.

Верхний уровень оперирует деревом ограничений (Constraint Tree, CT). Каждый узел CT содержит:

- набор ограничений $\mathcal{C} = \{C_1, C_2, \dots, C_k\}$, где C_i — множество ограничений для агента a_i ;
- набор путей $\Pi = \{\pi_1, \pi_2, \dots, \pi_k\}$, согласованных с ограничениями;
- суммарную стоимость $\text{cost} = \sum_{i=1}^k |\pi_i|$.

Корень дерева содержит пустые множества ограничений. На каждом шаге верхний уровень извлекает узел CT с минимальной стоимостью и проверяет наличие конфликтов в наборе путей. Если конфликтов нет, решение найдено.

При обнаружении конфликта между агентами a_i и a_j (вершинного: (a_i, a_j, v, t) , или рёберного: (a_i, a_j, u, v, t)) узел разветвляется на два дочерних

узла. В первом добавляется ограничение для агента a_i (запрет на нахождение в вершине v в момент t или на переход (u, v) в момент t), во втором — аналогичное ограничение для агента a_j .

Нижний уровень решает задачу поиска кратчайшего пути для одного агента с учётом заданного набора ограничений. В качестве нижеуровневого планировщика используется алгоритм A^* с расширением по времени, описанный выше.

CBS гарантирует оптимальность решения (по метрике SOC) и полноту. Вычислительная сложность в худшем случае экспоненциальна относительно числа агентов, однако на практике алгоритм эффективен для сценариев с умеренным числом конфликтов.

Вариант CBS с SIPP. В качестве альтернативного нижеуровневого планировщика может использоваться алгоритм SIPP (Safe Interval Path Planning), который оперирует безопасными временными интервалами — промежутками времени, в течение которых данная вершина свободна от ограничений. Состояние поиска определяется как (v, I) , где I — безопасный интервал для вершины v . Это позволяет существенно сократить пространство поиска по сравнению с time-expanded A^* при большом количестве ограничений, так как каждый интервал агрегирует множество временных шагов [15].

1.4.5 Enhanced CBS (ECBS)

Алгоритм ECBS (Enhanced CBS) является ограниченно субоптимальной модификацией CBS, обеспечивающей значительное ускорение за счёт ослабления требования оптимальности [16].

ECBS вводит параметр субоптимальности $w \geq 1$ и гарантирует, что стоимость найденного решения не превышает оптимальную более чем в w раз:

$$\text{cost}(\Pi_{\text{ECBS}}) \leq w \cdot \text{cost}(\Pi^*). \quad (7)$$

Ускорение достигается за счёт использования фокального поиска (focal search) на обоих уровнях алгоритма. На верхнем уровне поддерживается два списка открытых узлов: основной (OPEN), упорядоченный по стоимости, и фокальный (FOCAL), содержащий узлы со стоимостью не более $w \cdot f_{\min}$, где

$f_{\min}$ — минимальная стоимость в OPEN. Узлы из FOCAL выбираются по вторичному критерию, например, по числу конфликтов.

На нижнем уровне аналогично используется фокальный A*: среди путей, стоимость которых не превышает $w \cdot f_{\min}^{\text{low}}$, выбирается путь с минимальным числом конфликтов с уже построенными путями других агентов. Это приводит к построению путей, порождающих меньше конфликтов на верхнем уровне, что сокращает размер дерева ограничений.

1.4.6 Explicit Estimation CBS (EECBS)

Алгоритм EECBS (Explicit Estimation CBS) является дальнейшим развитием ECBS, повышающим эффективность фокального поиска на верхнем уровне за счёт использования онлайн-эвристики [11].

В ECBS выбор узлов из фокального списка основывается на числе конфликтов, которое лишь косвенно связано с итоговой стоимостью решения. EECBS вводит явную оценку стоимости разрешения конфликтов: для каждого узла СТ вычисляется $\hat{h}$ — оценка дополнительной стоимости, необходимой для устранения всех оставшихся конфликтов. Эта оценка обновляется в процессе поиска на основе информации, полученной при разветвлении узлов.

Узлы фокального списка упорядочиваются по $\hat{f} = g + \hat{h}$, что направляет поиск в сторону узлов, более вероятно приводящих к решению с низкой стоимостью. Граница субоптимальности w сохраняется, однако на практике EECBS находит решения значительно ближе к оптимуму, чем ECBS с тем же значением w .

1.4.7 M* (Subdimensional Expansion)

Алгоритм M* решает задачу MAPF путём поиска в совместном пространстве состояний, но в отличие от наивного подхода выполняет расширение в полное пространство состояний только при возникновении конфликтов [17].

В совместном пространстве состояний конфигурация системы из k агентов представляется как кортеж $(v_1, v_2, \dots, v_k) \in V^k$, а переход — как одновременное перемещение всех агентов. Наивный поиск в таком пространстве имеет сложность $O(|V|^k)$, что делает его непрактичным для большого числа агентов.

M^* использует принцип поддимерного расширения (subdimensional expansion): каждому агенту предварительно вычисляется индивидуальная оптимальная политика (кратчайший путь без учёта других агентов) с помощью обратного алгоритма Дейкстры от целевой вершины. При поиске в совместном пространстве каждый агент по умолчанию следует своей индивидуальной политике. Когда обнаруживается конфликт между агентами a_i и a_j в некоторой конфигурации, формируется множество столкновений (collision set), и конфигурация расширяется по всем возможным комбинациям действий конфликтующих агентов.

Множества столкновений обратно распространяются (back-propagated) к предшествующим конфигурациям, чтобы обеспечить полноту поиска. Алгоритм M^* является оптимальным и полным. Его эффективность определяется степенью связности конфликтов: если конфликты затрагивают лишь небольшие подмножества агентов, M^* работает значительно быстрее, чем поиск в полном совместном пространстве.

1.4.8 Cooperative A* (CA*)

Алгоритм Cooperative A* (CA*) реализует децентрализованный подход к решению задачи MAPF [18]. Агенты планируют пути последовательно, один за другим, с использованием общей таблицы резерваций.

Алгоритм работает следующим образом:

- 1) агенты упорядочиваются в некотором порядке (например, по приоритету);
- 2) для первого агента строится кратчайший путь с помощью A* с расширением по времени; его путь добавляется в таблицу резерваций;
- 3) для каждого следующего агента строится путь с учётом всех ранее зарезервированных позиций;
- 4) целевая позиция агента, достигшего цели, также резервируется на все последующие временные шаги.

Таблица резерваций содержит множество троек (v, t) , запрещающих нахождение очередного агента в вершине v в момент t . Помимо вершинных резерваций, учитываются и рёберные — запрет на переход по ребру (u, v) в момент t , если другой агент проходит по ребру (v, u) в тот же момент.

CA* не гарантирует оптимальности решения, поскольку результат

зависит от порядка планирования агентов: агенты, планирующие первыми, имеют больше свободы выбора пути. Более того, в некоторых конфигурациях СА* может не найти решение, даже если оно существует, если порядок агентов приводит к тупиковой ситуации. Тем не менее, СА* обладает низкой вычислительной сложностью — $O(k \cdot |V| \cdot T)$, где T — максимальная глубина по времени, — и хорошо масштабируется при большом числе агентов.

1.4.9 Windowed Hierarchical CA* (WHCA*)

Алгоритм WHCA* является модификацией СА*, в которой планирование ограничивается фиксированным временным окном W [18].

На каждой итерации WHCA* каждый агент планирует путь на ближайшие W временных шагов с использованием алгоритма СА* (последовательное планирование с таблицей резерваций). По истечении окна все агенты перепланируют пути от текущих позиций на следующие W шагов.

Ограничение горизонта планирования имеет два ключевых преимущества:

- уменьшается пространство поиска на каждой итерации, что ускоряет планирование;
- агенты, ещё не находящиеся вблизи друг друга, не создают избыточных ограничений.

Недостатком WHCA* является то, что локально оптимальные решения на каждом окне могут привести к глобально неоптимальным или даже тупиковым ситуациям. Размер окна W является параметром, определяющим баланс между качеством решения и скоростью планирования.

1.4.10 Priority Inheritance with Backtracking (PIBT)

Алгоритм PIBT представляет собой онлайн-децентрализованный алгоритм, в котором агенты планируют перемещение всего на один временной шаг вперёд [19]. Это делает PIBT самым «реактивным» из рассматриваемых алгоритмов.

Каждому агенту назначается приоритет, который определяет порядок планирования на текущем шаге. Приоритеты обновляются динамически: агент, не достигший цели, получает более высокий приоритет, чем агент, уже находящийся в целевой вершине.

Ключевым механизмом алгоритма является наследование приоритетов (priority inheritance): если агент a_i с высоким приоритетом хочет занять вершину, в которой находится агент a_j с более низким приоритетом, то a_j «наследует» приоритет a_i и вынужден переместиться. Если a_j не может найти свободную соседнюю вершину, инициируется откат (backtracking): a_i отменяет свой запрос и пробует другое направление.

Для выбора предпочтительного направления движения каждый агент использует таблицу расстояний (distance table) — матрицу кратчайших расстояний от каждой вершины до целевой, вычисленную предварительно обратным поиском в ширину.

PIBT не гарантирует оптимальности решения и полноты в общем случае, однако обеспечивает быстрое реактивное управление и хорошо масштабируется при большом числе агентов. Вычислительная сложность одного шага составляет $O(k)$, где k — число агентов.

1.5 Интеграция MRTA и MAPF

В реальных мульти-роботизированных системах задачи распределения и навигации решаются совместно: MRTA-алгоритм определяет, какие задачи выполняет каждый робот, а MAPF-алгоритм строит бесконфликтные маршруты к назначенным целям. Этот двухэтапный подход представляет собой основную архитектурную модель, реализованную в рассматриваемом программном комплексе.

При online-режиме назначения каждый робот получает одну задачу, после чего все активные роботы совместно планируют маршруты через MAPF-алгоритм. По завершении задачи роботом планировщик назначает ему следующую задачу, и маршрут для данного робота перестраивается с учётом оставшихся путей других агентов.

При queue-режиме каждому роботу сразу назначается полная цепочка задач. Начальные маршруты к первым задачам планируются совместно через выбранный MAPF-алгоритм. По мере завершения задач роботы переходят к следующим элементам цепочки, и маршруты достраиваются последовательно в стиле CA^* — с учётом путей остальных агентов.

Объективное сравнение различных комбинаций MRTA- и MAPF-алгоритмов при варьировании параметров среды (размеры карты, число роботов, плотность препятствий, стратегия размещения, источник

входных данных) является основной задачей разрабатываемого программного комплекса. Для проведения такого сравнения необходимо обеспечить, во-первых, воспроизводимость экспериментов на одних и тех же сценариях — это достигается использованием детерминированных генераторов псевдослучайных чисел и поддержкой эталонных наборов задач из библиотек Moving AI [12], — и, во-вторых, возможность изолированной оценки вклада каждой подсистемы, что обеспечивается, в частности, наличием случайного MRTA-распределителя, нейтрализующего влияние алгоритма распределения задач при сравнении MAPF-планировщиков.

2 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО КОМПЛЕКСА

2.1 Выбор стека технологий

При проектировании программного комплекса были выбраны следующие технологии:

- Python 3.12 — основной язык разработки, обеспечивающий высокую скорость прототипирования и обширную экосистему библиотек для научных вычислений;
- Pygame 2.6 — библиотека для создания графического интерфейса и визуализации, предоставляющая низкоуровневый доступ к 2D-рендерингу при минимальных зависимостях;
- NumPy — библиотека для работы с многомерными массивами, используемая для представления сеточной карты мира;
- Pandas — библиотека для обработки табличных данных, применяемая в подсистеме пакетного тестирования для агрегирования результатов экспериментов;
- Matplotlib — библиотека для построения графиков, используемая для автоматической генерации визуализаций результатов пакетных экспериментов;
- JSON — формат сериализации сценариев (карт), обеспечивающий совместимость и читаемость;
- PyInstaller — инструмент для сборки Python-приложения в автономный исполняемый файл, обеспечивающий кроссплатформенное распространение без необходимости установки интерпретатора Python.

Выбор Python обусловлен тем, что целью работы является сравнение алгоритмов, а не достижение максимальной производительности отдельных реализаций. Python позволяет реализовать все алгоритмы в единой среде с одинаковыми накладными расходами интерпретатора, что обеспечивает корректность сравнения. Библиотека Pygame была выбрана вместо веб-фреймворков или Qt, поскольку она обеспечивает прямой контроль над кадровым циклом, что критично для покадровой визуализации работы алгоритмов. Использование PyInstaller позволяет собирать приложение в единый исполняемый файл для операционных систем Windows, Linux и

macOS, что обеспечивает кроссплатформенность и упрощает распространение.

2.2 Архитектура программного комплекса

Архитектура программного комплекса построена по модульному принципу с чётким разделением ответственности между компонентами. На рисунке 1 представлена общая схема архитектуры.

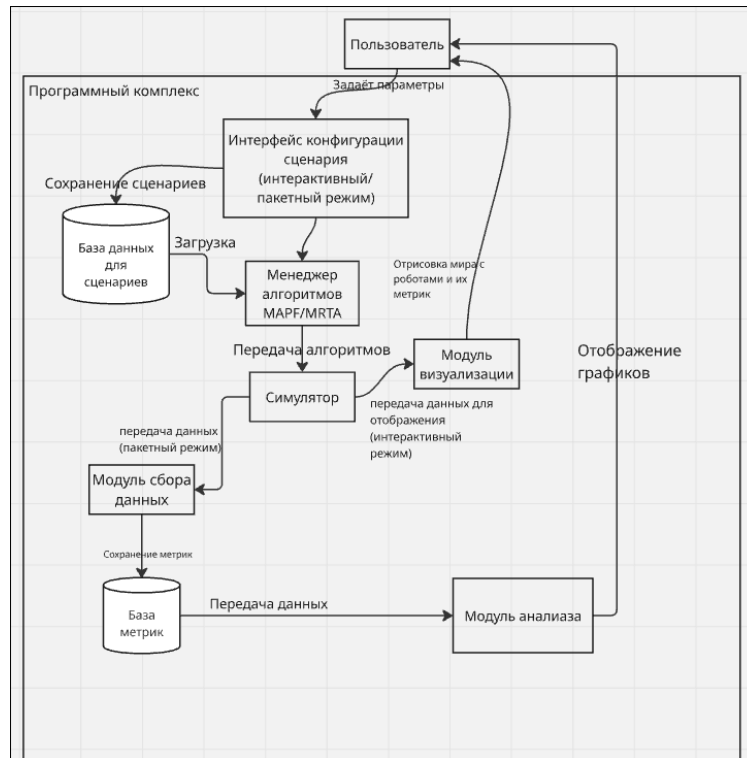


Рисунок 1 – Архитектура программного комплекса

Программный комплекс состоит из следующих модулей:

- core — базовые модели данных: мир (сеточная карта), агент, задача;
- mapf — реализации MAPF-алгоритмов, объединённые общим базовым классом;
- mrta — реализации MRTA-алгоритмов с единым интерфейсом;
- simulation — симулятор, координирующий планирование и пошаговое выполнение;
- batch — подсистема пакетного тестирования: конфигурация, генерация сценариев, запуск экспериментов, построение графиков;
- ui — экраны графического интерфейса: главное меню, редактор карт, выбор алгоритма, экраны батч-режима;

- visualization — отрисовка симуляции: карта, агенты, пути, панель метрик;
- registry — центральный реестр алгоритмов;
- scenario — модель сценария и сериализация в JSON.

Файловая структура проекта продемонстрирована на рисунке 2.

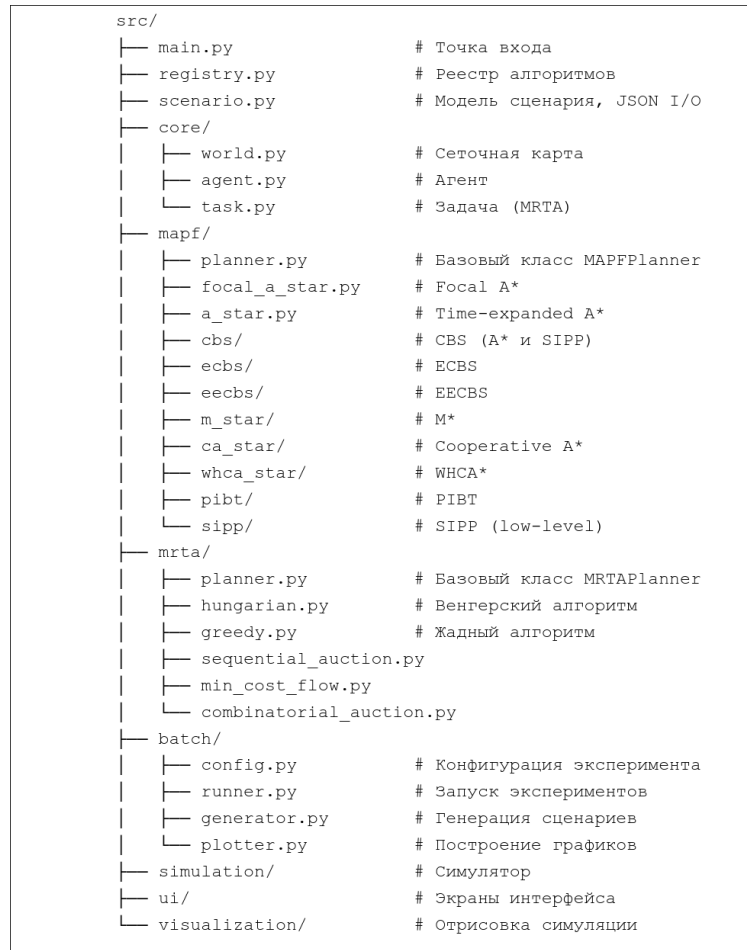


Рисунок 2 – Файловая структура проекта

2.3 Базовые классы алгоритмов

Ключевым архитектурным решением является использование паттерна «Стратегия» (Strategy) для подключения алгоритмов. Все MAPF- и MRTA-алгоритмы реализуют единый интерфейс, определённый базовыми классами MAPFPlanner и MRTAPlanner.

2.3.1 Класс MAPFPlanner

Базовый класс MAPFPlanner определяет контракт для всех алгоритмов поиска бесконфликтных маршрутов:

- метод `plan(world, agents)` принимает модель мира и список агентов, возвращает словарь вида `{agent_id: [(pos, time), ...]}` или `None` при неудаче;
- атрибуты класса содержат метаданные алгоритма: отображаемое имя (`DISPLAY_NAME`), тип архитектуры (`IS_CENTRALIZED`), режим работы (`IS_ONLINE`).

Метаданные используются интерфейсом для отображения информации об алгоритме и симулятором для выбора стратегии выполнения. Единый формат возвращаемого значения — список позиций с временными метками — обеспечивает взаимозаменяемость алгоритмов: любой MAPF-планировщик может быть использован в качестве навигационного модуля для MRTA без модификации кода симулятора.

2.3.2 Класс MRTAPlanner

Базовый класс `MRTAPlanner` определяет контракт для алгоритмов распределения задач:

- метод `plan(world, agents, tasks)` принимает мир, агентов и задачи, возвращает словарь `{agent_id: [Task, ...]}`;
- атрибут `ASSIGNMENT_MODE` определяет режим назначения: `"online"` или `"queue"`.

Режим назначения является ключевым проектным решением, определяющим поведение симулятора:

- в режиме `online` каждому агенту назначается по одной задаче; после её завершения планировщик вызывается повторно для освободившегося агента с оставшимися задачами;
- в режиме `queue` планировщик строит полную цепочку задач для каждого агента за один вызов; симулятор последовательно выполняет элементы цепочки без перепланирования.

Такое разделение позволяет реактивным алгоритмам (венгерский, жадный) адаптироваться к изменениям в реальном времени, а планирующим алгоритмам (аукционы, потоковые) оптимизировать глобальную стоимость назначения.

2.4 Модель данных

2.4.1 Класс World

Класс `World` представляет среду в виде двумерной сеточной карты. Внутреннее представление — массив NumPy размером $H \times W$, где значение 1 обозначает препятствие. Метод `is_free(x, y, time, reserved)` проверяет доступность клетки с учётом как статических препятствий, так и динамических резерваций (множество троек (x, y, t) , зарезервированных другими агентами).

2.4.2 Класс Agent

Класс `Agent` хранит текущую позицию агента (x, y) , целевую позицию $(goal_x, goal_y)$, построенный путь и индекс текущего шага. Метод `set_path` назначает агенту новый путь, `next_step` возвращает следующую позицию и продвигает индекс. Каждому агенту автоматически назначается уникальный цвет для визуализации.

2.4.3 Класс Task

Класс `Task` описывает задачу для MRTA-планировщика: уникальный идентификатор, целевая позиция (x, y) , приоритет, ссылка на назначенного агента и флаг завершения. Задачи создаются из целевых позиций сценария при выборе MRTA-режима.

2.4.4 Класс SceneData

Класс `SceneData` — алгоритмо-независимое описание сценария: размеры сетки, позиции препятствий, стартовые позиции агентов и целевые позиции. Один и тот же сценарий может использоваться как для MAPF (цели — конечные точки агентов), так и для MRTA (цели — задачи для распределения). Сериализация выполняется в формате JSON, что обеспечивает переносимость и возможность повторного использования сценариев.

2.5 Форматы входных файлов сценариев

Программный комплекс поддерживает три формата описания

сценариев, что обеспечивает совместимость как с внутренним представлением, так и с общепринятыми наборами тестовых данных из репозитория Moving AI Lab [12]. Загрузка выполняется единой функцией `load_map_file`, которая определяет формат по расширению файла и вызывает соответствующий парсер.

2.5.1 Внутренний формат JSON

Сценарии, созданные в редакторе карт программного комплекса, сохраняются в формате JSON. Файл содержит поля `grid_width`, `grid_height`, `obstacles`, `agent_starts` и `goals`, каждое из которых соответствует одноимённому атрибуту класса `SceneData`. Координаты препятствий, стартовых позиций и целей представлены массивами пар (x, y) . Формат легко читается и поддаётся ручному редактированию.

2.5.2 Формат Moving AI .map

Формат `.map` описывает только статическую структуру среды — размеры сетки и расположение препятствий. Файл начинается с заголовка, содержащего тип карты (`type`), высоту (`height`) и ширину (`width`), за которым следует ключевое слово `map` и далее — сетка символов размером $H \times W$. Каждый символ кодирует тип клетки: символы «.», «G» и «S» обозначают проходимые клетки, любые другие символы (в частности «@», «O», «T», «W») интерпретируются как препятствия. Стартовые позиции агентов и цели в формате `.map` не задаются, поэтому при импорте такого файла они должны быть сгенерированы программно либо загружены из связанного файла `.scen`.

2.5.3 Формат Moving AI .scen

Формат `.scen` описывает набор тестовых пар «старт–цель» для конкретной карты. Файл начинается со строки `version 1`, далее каждая строка содержит девять полей, разделённых табуляцией: номер класса сложности (`bucket`), имя файла карты, ширина и высота карты, координаты старта (s_x, s_y) , координаты цели (g_x, g_y) и оптимальная длина пути для одиночного агента. Пример строки:

В разработанном программном комплексе файл `.scen`

интерпретируется как *библиотека задач*: каждая строка задаёт один возможный сценарий движения для отдельного агента, а из множества таких задач при пакетном тестировании формируется требуемое число одновременно действующих агентов. Соответствующий файл `.map` (определяемый по полю имени карты в строках `.scen`) загружается автоматически и используется как источник информации о препятствиях; если файл карты находится в той же директории, что и файл сценариев, операция выполняется прозрачно для пользователя.

2.6 Система регистрации алгоритмов

Для обеспечения расширяемости системы реализован центральный реестр алгоритмов (модуль `registry`). Реестр представляет собой словарь, содержащий экземпляры всех доступных планировщиков, сгруппированные по типу ("MAPF" и "MRTA").

Функция `get_planner(algo_type, display_name)` выполняет поиск планировщика по типу и отображаемому имени. Такая архитектура позволяет добавлять новые алгоритмы без модификации существующего кода: достаточно реализовать подкласс `MAPFPlanner` или `MRTAPlanner`, импортировать его в `registry.py` и добавить экземпляр в соответствующий список.

Реестр используется как интерактивным интерфейсом (для формирования списка алгоритмов на экране выбора), так и подсистемой пакетного тестирования (для перебора комбинаций алгоритмов).

2.7 Симулятор: планирование и выполнение

Класс `Simulator` является центральным координирующим компонентом, объединяющим планирование и пошаговое выполнение для обоих режимов работы.

2.7.1 Этап планирования

Метод `plan()` выполняет полный цикл планирования с замером времени и памяти (`tracemalloc`):

- 1) В режиме MAPF: вызывается `planner.plan(world, agents)`, полученные пути назначаются агентам.

2) В режиме MRTA: процесс двухэтапный:

- MRTA-планировщик распределяет задачи:
`planner.plan(world, agents, tasks);`
- навигационный MAPF-планировщик строит маршруты к первым задачам: `_nav_all(agents)`.

Метод `_nav_all` является ключевым связующим элементом между MRTA и MAPF. Если задан навигационный планировщик (`nav_planner`), он вызывается для совместного построения бесконфликтных путей всех агентов. В противном случае используется последовательный fallback в стиле СА*: агенты планируются один за другим, каждый — с учётом путей предыдущих через общую таблицу резерваций.

2.7.2 Этап выполнения

Метод `step()` выполняет один временной шаг симуляции:

- 1) каждый агент, имеющий невыполненный путь, перемещается на следующую позицию;
- 2) обновляется метрика SOC (каждое перемещение увеличивает счётчик на 1);
- 3) в режиме MRTA проверяется завершение задач: если агент достиг позиции назначенной задачи, задача помечается как завершённая;
- 4) при завершении задачи в queue-режиме из очереди извлекается следующая задача, и маршрут достраивается методом `_path_to_single`;
- 5) при завершении задачи в online-режиме вызывается повторное планирование для освободившегося агента.

2.7.3 Сбор метрик

Симулятор собирает шесть метрик, разделяя время планирования на составляющие:

Таблица 2 – Метрики, собираемые симулятором

Метрика	Описание
makespan	Общее число шагов до завершения всех задач
soc	Sum of Costs — суммарное число перемещений всех агентов
computation_s	Суммарное время планирования (секунды)
plan_mrta_s	Время, затраченное на MRTA-назначение
plan_mapf_s	Время, затраченное на навигационное планирование
memory_peak_mb	Пиковое потребление памяти (МБ, через tracemalloc)

2.8 Подсистема пакетного тестирования

Подсистема пакетного тестирования обеспечивает автоматизированное проведение серий экспериментов с различными комбинациями алгоритмов и параметров среды.

2.8.1 Конфигурация эксперимента

Класс `BatchConfig` определяет все параметры пакетного эксперимента:

- списки выбранных MAPF- и MRTA-алгоритмов;
- диапазон числа роботов: минимум, максимум, шаг;
- параметры процедурной генерации карты: размеры сетки, плотность препятствий, стратегия размещения, флаг проверки достижимости;
- число сценариев на каждое значение числа роботов (для статистической достоверности);
- список импортированных файлов карт в форматах JSON и `.map` (`imported_maps`);
- список файлов `.scen` для использования в качестве источника пар «старт–цель» (`scen_files`);
- таймаут на этап планирования и таймаут на этап симуляции;
- путь к директории вывода.

Поля `imported_maps` и `scen_files` являются взаимоисключающими: при заполнении одного из них другое сбрасывается

интерфейсом, а параметры процедурной генерации игнорируются.

2.8.2 Генерация сценариев

Класс `SceneGenerator` создаёт сценарии для экспериментов. Источник сценария выбирается по приоритету: при наличии файлов `.scen` они используются в первую очередь; при наличии импортированных карт сценарий формируется на их основе; в противном случае выполняется процедурная генерация.

Процедурная генерация поддерживает четыре стратегии размещения агентов и задач:

- `uniform` — агенты и задачи размещаются на случайных свободных клетках равномерно;
- `clustered` — агенты концентрируются в центре карты, задачи — у периметра;
- `counter` — агенты слева, задачи справа (встречные потоки);
- `min_dist` — гарантируется минимальное расстояние между стартовыми позициями.

Генератор выполняет проверку достижимости каждой пары «старт–цель» с помощью поиска в ширину (BFS) и повторяет генерацию при обнаружении недостижимых целей (до 20 попыток).

2.8.3 Использование файлов `.scen` в пакетном режиме

Для интеграции файлов формата `.scen` в пакетный режим реализован класс `ScenLibrary`, представляющий собой кэшированное в памяти описание загруженного файла. При создании библиотеки файл `.scen` разбирается единожды, после чего сохраняются размер сетки, список препятствий и полный набор пар «старт–цель». Внутри `SceneGenerator` поддерживается словарь `_scen_cache`, ставящий в соответствие пути к файлам уже загруженные библиотеки, что исключает повторное чтение при многократном обращении в течение одного эксперимента.

Метод `sample(n_robots, seed)` формирует сценарий для требуемого числа агентов из библиотеки следующим образом:

- 1) на основе значения `seed` создаётся независимый генератор псевдослучайных чисел;

- 2) копия списка пар «старт–цель» перемешивается этим генератором;
- 3) перемешанный список последовательно сканируется; пара включается в сценарий, если её стартовая позиция ещё не использована;
- 4) процесс завершается, когда набрано `n_robots` пар с попарно различными стартовыми позициями.

Дедупликация выполняется только по стартовым позициям, поскольку совпадение целевых позиций двух агентов допустимо. Если в библиотеке недостаточно различных стартовых позиций для требуемого числа агентов, сценарий помечается как несформированный.

2.8.4 Запуск и вывод результатов

Класс `BatchRunner` перебирает все комбинации (MRTA-алгоритм × MAPF-алгоритм × число роботов × seed сценария) и для каждой создаёт экземпляр симулятора. Таймауты реализованы через `threading.Timer`: при превышении установленного времени на этап планирования или этап симуляции прогон помечается как неуспешный.

Результаты агрегируются в табличный объект `pandas.DataFrame` и экспортируются модулем `plotter`:

- CSV-файл со всеми метриками каждого прогона. Состав колонок приведён в таблице 3;
- четыре графика (Matplotlib): Makespan, SOC, время планирования и доля успешных прогонов (Success Rate) в зависимости от числа роботов.

Таблица 3 – Состав CSV-файла результатов пакетного эксперимента

Колонка	Назначение
mrta_algo	Имя MRTA-алгоритма
mapf_algo	Имя MAPF-алгоритма (навигатор)
n_robots	Число агентов в прогоне
scenario_seed	Seed сценария
map_width	Ширина карты, клеток
map_height	Высота карты, клеток
obstacle_density	Доля клеток, занятых препятствиями
placement	Стратегия размещения или признак импорта
scene_source	Источник сценария: random, imported или scen:<имя_файла>
success	Признак успешного завершения прогона
makespan	Время завершения миссии всеми агентами
soc	Sum of Costs — суммарное число перемещений
plan_total_s	Суммарное время планирования, с
plan_mrta_s	Время MRTA-назначения, с
plan_mapf_s	Время навигационного планирования, с
memory_mb	Пиковое потребление памяти, МБ

Вывод сохраняется в каталог с временной меткой вида batch_results/YYYYMMDD_NHMMSS/. Колонка scene_source позволяет на этапе анализа отделять результаты, полученные на процедурно сгенерированных картах, от результатов, полученных на эталонных файлах из набора Moving AI.

3 РЕАЛИЗАЦИЯ И РЕЗУЛЬТАТЫ

3.1 Пользовательский интерфейс

Графический интерфейс программного комплекса построен на базе Pygame и состоит из пяти основных экранов, образующих последовательный рабочий поток: главное меню, редактор карт, выбор алгоритма, визуализация симуляции и экраны батч-режима. Навигация между экранами осуществляется клавишей ESC (возврат на предыдущий экран) и кнопками действий.

3.1.1 Главное меню

Главное меню предоставляет выбор между тремя режимами работы:

- Create — создание нового сценария в редакторе карт;
- Load — загрузка ранее сохранённого сценария из файла через системный диалог; поддерживаются внутренний формат JSON, формат .map и формат .scen библиотеки Moving AI;
- Batch Mode — переход к автоматизированному пакетному тестированию.

На рисунке 3 представлен внешний вид главного меню.



Рисунок 3 – Главное меню программного комплекса

3.1.2 Редактор карт

Редактор карт позволяет интерактивно создавать и модифицировать сценарии. Левая часть экрана содержит сеточную карту, правая — панель инструментов. Пользователь может:

- рисовать и удалять препятствия щелчками мыши;
- размещать стартовые позиции агентов и целевые позиции;
- изменять размер сетки непосредственно в панели настроек;
- сохранять и загружать сценарии в формате JSON.

На рисунке 4 представлен внешний вид редактора карт.



Рисунок 4 – Редактор карт с размещёнными агентами и целями

3.1.3 Экран выбора алгоритма

На экране выбора алгоритма отображается предварительный просмотр карты и панель с двумя вкладками: MAPF и MRTA. Пользователь выбирает алгоритм из доступного списка, после чего отображаются его характеристики (тип архитектуры, режим работы). При выборе MRTA-алгоритма дополнительно предлагается выбрать MAPF-алгоритм в качестве навигационного планировщика. Перед запуском выполняется валидация: проверяется, что число агентов и целей совместимо с выбранным типом алгоритма.

3.1.4 Визуализация симуляции

Экран симуляции является центральным элементом интерактивного режима. Левая часть отображает сеточную карту с агентами, целями (или задачами в режиме MRTA), препятствиями и опционально — построенными путями агентов. Правая боковая панель содержит:

- информацию об алгоритме (имя, тип, режим);
- текущий статус (PLANNING, RUNNING, DONE, FAILED);
- метрики в реальном времени: Makespan, SOC, время планирования (общее, MRTA, MAPF), память;
- управление: пауза (Space), перезапуск (R), переключение отображения путей (P), регулировка скорости (+/-).

Агенты отображаются цветными квадратами с номерами, цели в режиме MAPF — перекрестиями соответствующего цвета, задачи в режиме MRTA — ромбами с номерами. Пути отображаются в виде цветных точек, позволяя визуально оценить маршруты и выявить потенциальные зоны скопления.

На рисунке 5 представлен внешний вид процесса визуализации симуляции.



Рисунок 5 – Визуализация симуляции (MAPF-режим, алгоритм CBS)

3.1.5 Батч-режим

Батч-режим реализован в виде последовательности из трёх экранов:

- 1) Выбор алгоритмов — флажки для MAPF- и MRTA-алгоритмов с

- отображением общего числа комбинаций;
- 2) Настройка параметров — размеры сетки, диапазон числа роботов (минимум, максимум, шаг), плотность препятствий, число сценариев, стратегия размещения, таймауты на планирование и симуляцию, кнопка «Import Maps» для загрузки карт в форматах JSON или .map и кнопка «Import .scen» для загрузки файлов сценариев Moving AI; при выборе файлов .scen поля процедурной генерации (размеры сетки, плотность препятствий, стратегия размещения, проверка достижимости) автоматически блокируются и затемняются, поскольку соответствующие параметры берутся из загруженного файла;
 - 3) Прогресс — индикатор выполнения в реальном времени (эксперименты выполняются в отдельном потоке), текущая комбинация, кнопка открытия каталога с результатами.

На рисунке 6 представлен внешний вид настройки параметров пакетного тестирования.

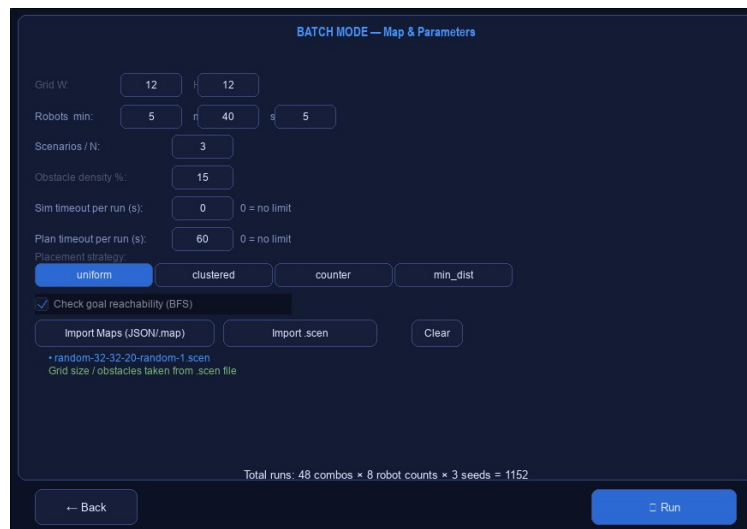


Рисунок 6 – Экран настройки параметров пакетного тестирования

3.2 Сравнительный анализ алгоритмов

3.2.1 Методика эксперимента

Для объективного сравнения реализованных алгоритмов были проведены серии экспериментов с использованием батч-режима. В качестве источника сценариев использовался готовый файл

random-32-32-20-random-1.scen из набора Moving AI [12], описывающий карту размером 32×32 клетки с плотностью препятствий около 20%. Стартовые позиции и цели агентов для каждого запуска выбирались из библиотеки задач этого файла детерминированным образом в соответствии со значением seed. Параметры экспериментов:

- карта: random-32-32-20-random-1.scen (32×32 , плотность препятствий $\approx 20\%$);
- диапазон числа роботов: от 5 до 40 с шагом 5;
- число сценариев на каждое значение числа роботов: 3;
- таймаут на этап планирования: 60 секунд;
- тестируемые MAPF-алгоритмы: CBS (A*), CBS (SIPP), ECBS, EECBS, M*, CA*, WHCA*, PIBT;
- тестируемые MRTA-алгоритмы: венгерский, жадный, Min-Cost Flow, последовательный аукцион, комбинаторный аукцион, случайный.

Для каждой комбинации (MRTA-алгоритм $\times$ MAPF-алгоритм $\times$ число роботов $\times$ seed) измерялись: makespan, SOC, суммарное время планирования и флаг успешного завершения. Результаты усреднялись по всем сценариям для каждого значения числа роботов. Всего выполнено 1152 запусков.

3.2.2 Анализ MAPF-алгоритмов

Результаты экспериментов позволяют выявить характерные особенности каждого класса MAPF-алгоритмов.

Оптимальные алгоритмы (CBS, M*). CBS (A*) обеспечивает наименьший SOC среди всех алгоритмов: при 40 роботах средний SOC по успешным прогонам составляет около 241 шага против 948 у PIBT. При этом среднее время планирования CBS (A*) оказывается сопоставимым с PIBT и меньше, чем у CA* и WHCA*, что объясняется тем, что децентрализованные алгоритмы в online-режиме MRTA вызываются повторно при каждом завершении задачи, накапливая суммарное время. Вариант CBS (SIPP) в среднем медленнее CBS (A*) при данном наборе сценариев, хотя теоретически должен выигрывать при большом числе ограничений.

M* в условиях плотной карты с большим числом агентов показывает наихудший success rate (80.6%) и аномально высокий SOC: при 10 роботах средний SOC составляет около 231 шага, тогда как CBS (A*) даёт около 120. Это соответствует теоретическому ожиданию: при высокой плотности

конфликтов M^* деградирует до поиска в полном совместном пространстве состояний.

Субоптимальные алгоритмы (ECBS, EECBS). ECBS и EECBS дают SOC, близкий к CBS, при несколько меньшем числе успешных прогонов. EECBS незначительно уступает ECBS по среднему SOC на данном наборе сценариев. Оба алгоритма представляют хороший компромисс между качеством решения и надёжностью завершения для сценариев с 10–25 агентами.

Децентрализованные алгоритмы (CA^* , WHCA*, PIBT). PIBT единственный из всех алгоритмов обеспечивает стопроцентный success rate при любом числе агентов вплоть до 40. Однако его SOC является наибольшим: при 40 роботах среднее значение превышает 948 шагов, что примерно в четыре раза больше, чем у CBS (A^*). CA^* и WHCA* дают умеренный SOC, близкий к субоптимальным алгоритмам, однако их суммарное время планирования максимально среди всех алгоритмов (среднее 5.4 и 6.2 с соответственно) вследствие многократных перепланирований в online-режиме MRTA.

В таблице 4 приведена сравнительная характеристика MAPF-алгоритмов по результатам экспериментов.

Таблица 4 – Сравнительная характеристика MAPF-алгоритмов

Алгоритм	SOC	Success rate	Область применения
CBS (A*)	отличный	высокий	Задачи, где критично качество маршрутов; конкурентен по времени при online-MRTA
CBS (SIPP)	отличный	средний	Сценарии с плотными ограничениями по времени
ECBS	хороший	высокий	Среднее число агентов, допустима ограниченная субоптимальность
EECBS	хороший	высокий	Аналогично ECBS
M*	плохой	низкий	Разреженные конфликты; не рекомендуется при плотных картах
CA*	хороший	высокий	Большое число агентов при условии offline-MRTA
WHCA*	хороший	высокий	Аналогично CA*
PIBT	плохой	наивысший	Задачи, где требуется гарантированное завершение при любом числе агентов

Таблица 5 – Сравнительная характеристика алгоритмов MRTA

Алгоритм	Режим	Когда применять
Венгерский	online	мало роботов, непредсказуемые задачи
Жадный	online	реальное время, ограниченные ресурсы
Мин. стоимость потока	queue	много задач, глобальная оптимизация
Последовательный аукцион	queue	компромисс скорости и качества
Комбинаторный аукцион	queue	когда задачи расположены группами

3.2.3 Анализ MRTA-алгоритмов

Анализ MRTA-алгоритмов выявляет различия между online- и queue-подходами.

Online-алгоритмы (венгерский, жадный). Венгерский алгоритм обеспечивает оптимальное решение задачи назначения на каждой итерации, что приводит к лучшему суммарному расстоянию при небольшом числе задач.

Жадный алгоритм значительно быстрее, но может давать существенно худшие назначения, особенно когда расположение задач неоднородно. Недостатком online-подхода является отсутствие глобальной оптимизации: каждое назначение учитывает только текущее состояние.

Queue-алгоритмы (последовательный аукцион, минимальная стоимость потока, комбинаторный аукцион). Алгоритмы с построением цепочек учитывают все задачи одновременно, что позволяет минимизировать суммарные перемещения. Минимальная стоимость потока обеспечивает оптимальное решение на каждом слое назначений. Последовательный аукцион даёт хороший баланс скорости и качества. Комбинаторный аукцион с локальной оптимизацией обменом показывает лучшие результаты в сценариях, где существуют естественные кластеры задач, поскольку может назначить близкие задачи одному роботу.

Случайное распределение задач. Алгоритм случайного распределения не выполняет оптимизации и заведомо уступает остальным MRTA-алгоритмам по суммарной стоимости назначения. Его роль в проведённых экспериментах — роль базовой линии (baseline), относительно которой оценивается выигрыш от применения оптимизационных и эвристических методов. Кроме того, случайное распределение позволяет проводить чистое сравнение MAPF-планировщиков: при фиксированном (детерминированном по seed) случайном назначении расхождения в метриках обусловлены исключительно качеством навигационного планирования.

Выбор алгоритма MRTA существенно влияет на итоговый SOC и makespan, однако зависит также от выбранного MAPF-планировщика: оптимальное назначение не гарантирует оптимальных маршрутов, и наоборот.

3.2.4 Результаты пакетного тестирования

На рисунках 7–10 представлены графики зависимости ключевых метрик от числа роботов, полученные в ходе пакетного тестирования.

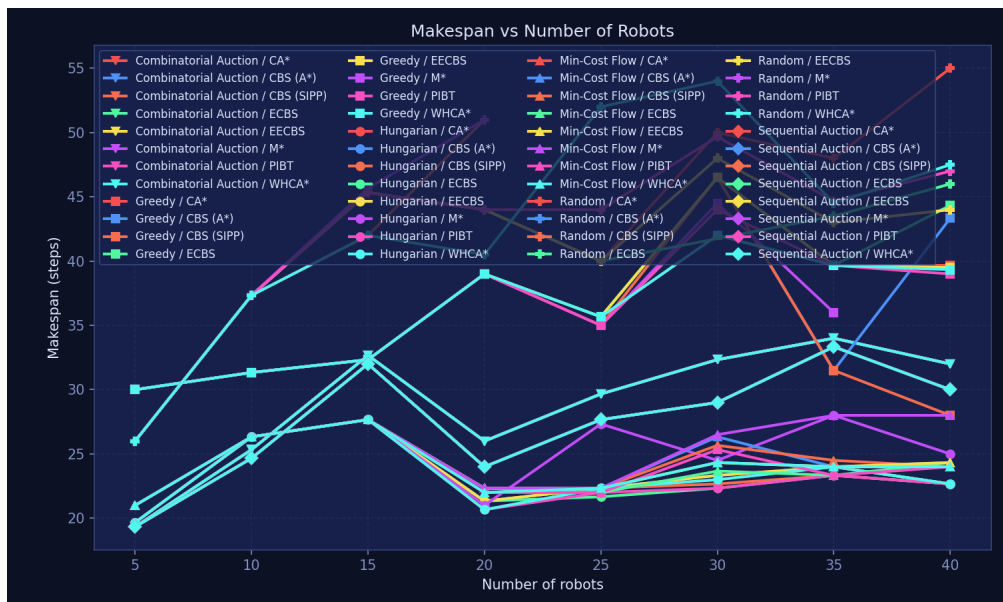


Рисунок 7 – Зависимость makespan от числа роботов

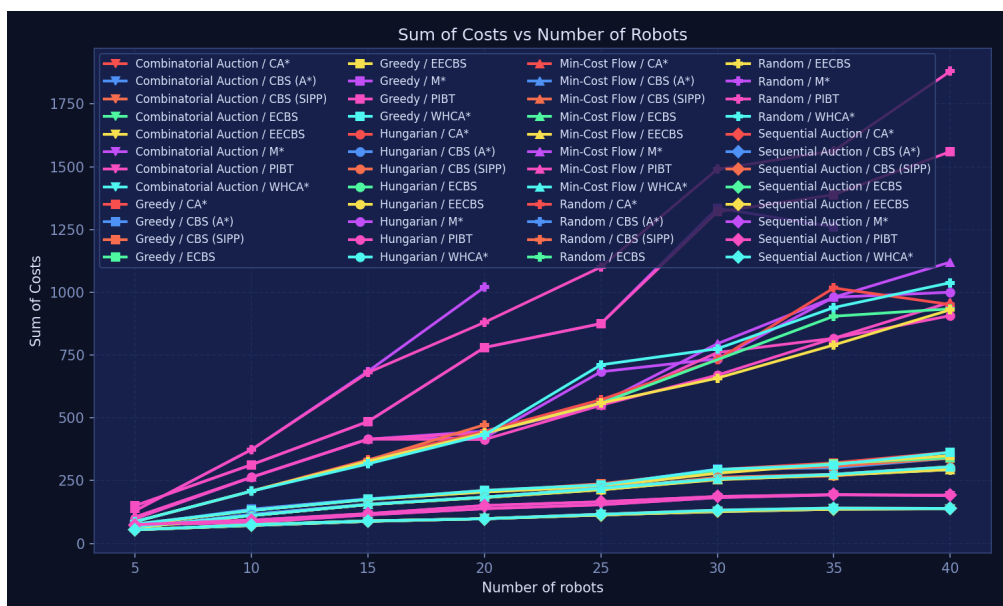


Рисунок 8 – Зависимость SOC от числа роботов

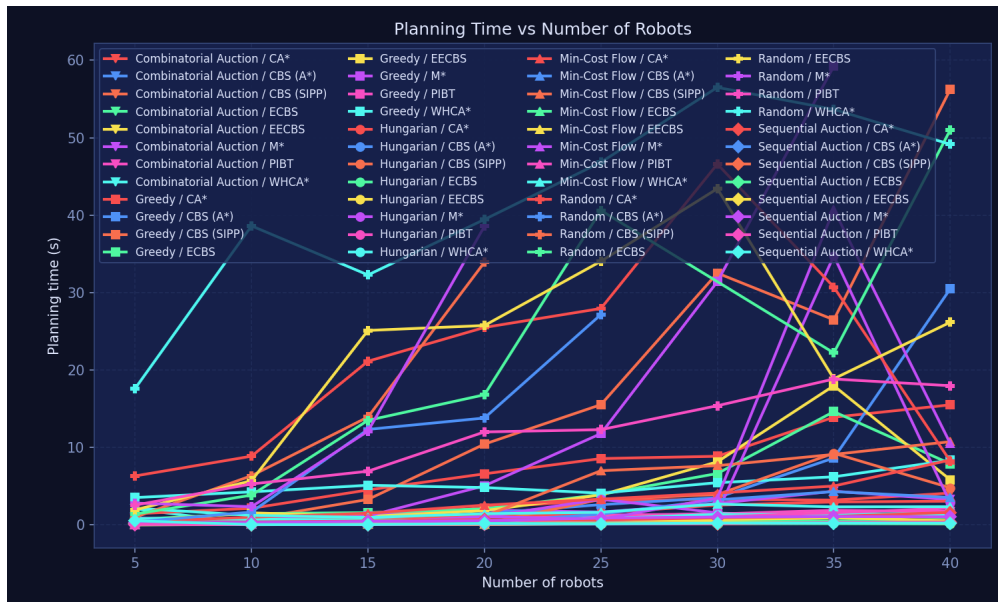


Рисунок 9 – Зависимость времени планирования от числа роботов

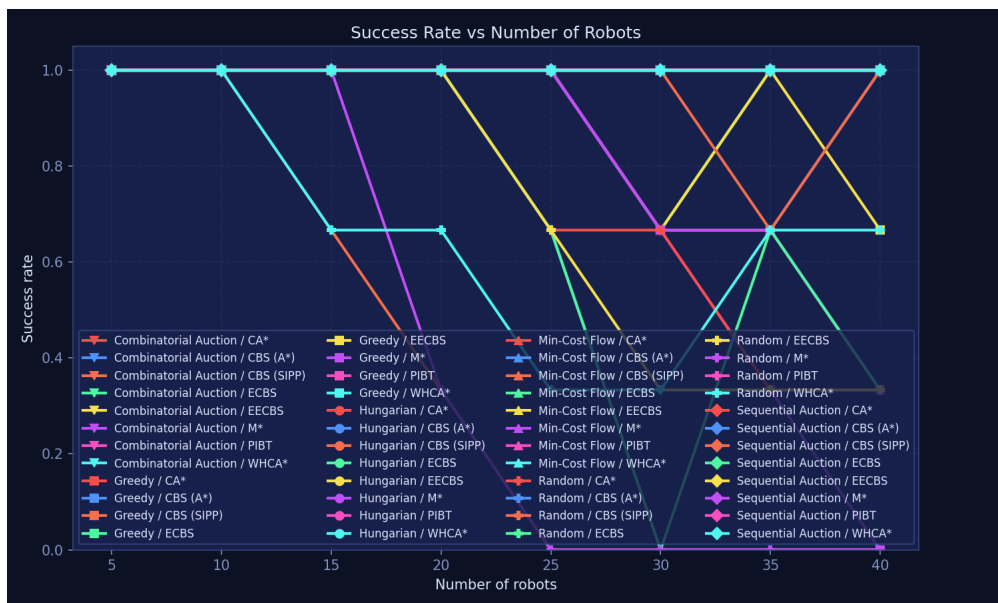


Рисунок 10 – Зависимость доли успешных запусков от числа роботов

С увеличением числа роботов наблюдаются следующие закономерности:

- 1) Makespan на данной карте растёт умеренно (с ≈ 22 шагов при 5 роботах до ≈ 32 при 40): карта 32×32 предоставляет достаточно пространства для обходных маршрутов. Алгоритмы CBS (SIPP) и WHCA* дают несколько меньший makespan при большом числе агентов, PIBT — несколько больший;
- 2) SOC является наиболее дифференцирующей метрикой: CBS (A*),

CBS (SIPP), ECBS и EECBS обеспечивают наименьший суммарный пробег; M^* и PIBT генерируют существенно более длинные маршруты (в 3–4 раза при 40 агентах) вследствие архитектурных особенностей этих алгоритмов;

- 3) Суммарное время планирования наибольшее у CA^* и WHCA* из-за многократных перезапусков в online-режиме MRTA; PIBT и CBS (A^*) в среднем завершают планирование быстрее всего; M^* и CBS (SIPP) чаще упираются в таймаут при 30–40 агентах;
- 4) Success Rate снижается с ростом числа агентов для всех алгоритмов, кроме PIBT, который сохраняет 100% во всём диапазоне. Наихудший результат демонстрирует M^* (80.6% в среднем), что подтверждает его нецелесообразность на плотных картах при большом числе агентов.

3.3 Обсуждение результатов

Проведённый анализ подтверждает, что не существует универсально лучшего алгоритма: выбор зависит от условий среды и приоритетов системы.

Для задач, где критично минимальное суммарное перемещение агентов (SOC), рекомендуется CBS (A^*) или CBS (SIPP): они обеспечивают наилучшее качество маршрутов при приемлемом времени планирования в режиме online-MRTA. Для сценариев, где требуется гарантированное завершение при большом числе агентов (15–40), единственным надёжным выбором является PIBT, однако за счёт существенно худшего SOC. Алгоритмы ECBS и EECBS занимают промежуточное положение. M^* не рекомендуется для плотных карт с большим числом агентов. CA^* и WHCA* целесообразнее применять с queue-режимом MRTA, когда повторные перепланирования минимальны.

При комбинировании MRTA и MAPF алгоритмы с построением цепочек (queue-режим) показывают преимущество при большом числе задач, поскольку минимизируют холостые перемещения роботов. Однако online-подход (венгерский алгоритм) более устойчив к непредвиденным ситуациям, поскольку перепланирует назначения по мере завершения задач.

Разработанный программный комплекс позволяет проводить подобный анализ для произвольных конфигураций среды и готовых наборов сценариев, что даёт возможность обоснованно выбирать алгоритмы для конкретных

прикладных задач.

ЗАКЛЮЧЕНИЕ

В ходе выполнения данной работы был разработан кроссплатформенный программный комплекс для тестирования и сравнения эффективности алгоритмов управления мульти-роботизированными системами. Поставленная цель достигнута в полном объеме, все сформулированные задачи решены.

В рамках работы выполнено следующее:

- 1) проведён анализ существующих алгоритмов решения задач MAPF и MRTA, выполнена их классификация по ключевым критериям: архитектуре управления, оптимальности и режиму работы;
- 2) разработана модульная архитектура программного комплекса на основе паттерна «Стратегия», обеспечивающая расширяемость — добавление новых алгоритмов без модификации существующего кода;
- 3) реализованы восемь MAPF-алгоритмов (CBS, CBS-SIPP, ECBS, EECBS, M*, CA*, WHCA*, PIBT), охватывающих оптимальные, субоптимальные и эвристические подходы;
- 4) реализованы пять MRTA-алгоритмов (венгерский, жадный, последовательный аукцион, минимальная стоимость потока, комбинаторный аукцион) в двух режимах назначения: online и queue;
- 5) разработан графический интерфейс, включающий редактор карт с возможностью сохранения и загрузки сценариев в формате JSON, экран выбора алгоритмов и визуализацию симуляции с отображением метрик в реальном времени;
- 6) реализован режим пакетного тестирования с автоматической генерацией сценариев, четырьмя стратегиями размещения агентов, контролем таймаутов и экспортом результатов в CSV и графики;
- 7) обеспечена кроссплатформенность: приложение собирается в автономный исполняемый файл с помощью PyInstaller для Windows, Linux и macOS;
- 8) проведён сравнительный анализ реализованных алгоритмов, выявивший характерные области применимости каждого из них.

Сравнительный анализ показал, что оптимальные алгоритмы (CBS, M*)

обеспечивают наилучшее качество решений при малом числе агентов, субоптимальные (ECBS, EECBS) — оптимальный баланс скорости и качества для средних сценариев, а децентрализованные (CA*, WHCA*, PIBT) — наилучшую масштабируемость при большом числе агентов. Разработанный программный комплекс позволяет воспроизводить подобный анализ для произвольных конфигураций среды, что делает его полезным инструментом для обоснованного выбора алгоритмов в конкретных прикладных задачах.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Wurman P. R., D'Andrea R., Mountz M. Coordinating hundreds of cooperative, autonomous vehicles in warehouses // *AI Magazine*. — 2008. — Т. 29, № 1. — С. 9—20.
2. Parker L. E. Multiple mobile robot systems // *Springer Handbook of Robotics*. — Springer, 2008. — С. 921—941.
3. Multi-agent pathfinding: Definitions, variants, and benchmarks / R. Stern [и др.] // *Proceedings of the International Symposium on Combinatorial Search (SoCS)*. — 2019. — С. 151—158.
4. Gerkey B. P., Mataric M. J. A formal analysis and taxonomy of task allocation in multi-robot systems // *International Journal of Robotics Research*. — 2004. — Т. 23, № 9. — С. 939—954.
5. LaValle S. M. *Planning Algorithms*. — Cambridge University Press, 2006.
6. Korsah G. A., Stentz A., Dias M. B. A comprehensive taxonomy for multi-robot task allocation // *International Journal of Robotics Research*. — 2013. — Т. 32, № 12. — С. 1495—1512.
7. Market-based multirobot coordination: A survey and analysis / M. B. Dias [и др.] // *Proceedings of the IEEE*. — 2006. — Т. 94, № 7. — С. 1257—1270.
8. Kuhn H. W. The Hungarian method for the assignment problem // *Naval Research Logistics Quarterly*. — 1955. — Т. 2, № 1/2. — С. 83—97.
9. Ahuja R. K., Magnanti T. L., Orlin J. B. *Network Flows: Theory, Algorithms, and Applications*. — Prentice Hall, 1993.
10. Conflict-based search for optimal multi-agent pathfinding / G. Sharon [и др.] // *Artificial Intelligence*. — 2015. — Т. 219. — С. 40—66.
11. Li J., Ruml W., Koenig S. EECBS: A bounded-suboptimal search for multi-agent path finding // *Proceedings of the AAAI Conference on Artificial Intelligence*. Т. 35. — 2021. — С. 12353—12362.
12. Sturtevant N. R. Benchmarks for grid-based pathfinding // *IEEE Transactions on Computational Intelligence and AI in Games*. — 2012. — Т. 4, № 2. — С. 144—148.

13. Yu J., LaValle S. M. Structure and intractability of optimal multi-robot path planning on graphs // Proceedings of the AAAI Conference on Artificial Intelligence. — 2013. — C. 1443—1449.
14. Hart P. E., Nilsson N. J., Raphael B. A formal basis for the heuristic determination of minimum cost paths // IEEE Transactions on Systems Science and Cybernetics. — 1968. — T. 4, № 2. — C. 100—107.
15. Phillips M., Likhachev M. SIPP: Safe interval path planning for dynamic environments // IEEE International Conference on Robotics and Automation (ICRA). — IEEE. 2011. — C. 5628—5635.
16. Suboptimal variants of the conflict-based search algorithm for the multi-agent pathfinding problem / M. Barer [и др.] // Proceedings of the International Symposium on Combinatorial Search (SoCS). — 2014. — C. 19—27.
17. Wagner G., Choset H. Subdimensional expansion for multirobot path planning // Artificial Intelligence. — 2015. — T. 219. — C. 1—24.
18. Silver D. Cooperative pathfinding // Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE). — 2005. — C. 117—122.
19. Priority inheritance with backtracking for iterative multi-agent path finding / K. Okumura [и др.] // Artificial Intelligence. — 2022. — T. 310. — C. 103752.

ПРИЛОЖЕНИЕ А

Репозиторий проекта

Перейти в репозиторий можно по QR-коду на рисунке А.1.



Рисунок А.1 – QR-код ссылки на репозиторий GitHub